14 INTERFACE vs DTO — SIDE BY SIDE

Both describe "the shape of an object." That's why they get confused. The difference isn't in what they look like — it's in WHAT DATA they describe and WHEN that data gets checked.

interface Task	→ shape of data ALREADY INSIDE your app (trusted, compile-time only)
class CreateTaskDto	→ shape of data ARRIVING FROM OUTSIDE your app (untrusted, checked at runtime)

REAL EXAMPLE FROM THIS PROJECT

```
// src/task/interfaces/task.interface.ts
export interface Task {
  id: number;
  title: string;
  description?: string;
  done: boolean;
}

// src/task/dto/create-task.dto.ts
import { IsNotEmpty, IsOptional, IsString } from 'class-validator';

export class CreateTaskDto {
  @IsString()
  @IsNotEmpty()
  title: string | undefined;

  @IsOptional()
  @IsString()
  description?: string;
}
```

Notice CreateTaskDto doesn't even have `id` or `done` — a client creating a task shouldn't get to choose its `id` or mark it `done` on creation; the SERVER decides those (see task.service.ts `create()`). That's a DTO doing its job: describing exactly what's ALLOWED to come in, not mirroring the full internal model.

THE CORE DIFFERENCE: WHEN IS IT CHECKED?

interface Task - Checked ONLY by the TypeScript compiler, ONLY while you're writing/building code. - Completely erased once compiled to JavaScript — at runtime, there is no `Task`, no check, nothing. It never existed as far as Node.js is concerned. - So it can only protect you from mistakes YOU make in your own code (e.g. assigning a number to `title`). It cannot protect you from a malicious or buggy HTTP client.

```
class CreateTaskDto
  - Also checked by the compiler, same as any class.
  - BUT it additionally survives into the compiled JavaScript as a
    real class, decorated with class-validator rules
    (@IsString(), @IsNotEmpty()).
  - Nest's ValidationPipe (wired up in main.ts via
    `app.useGlobalPipes(new ValidationPipe())`) uses those decorators
```

```
    to check the ACTUAL JSON body sent by a real HTTP request, live,
    every time — something no interface could ever do, because an
    interface isn't there anymore at that point.
```

Try it: POST /task with {"title": 123} gets rejected with a 400 by CreateTaskDto's ValidationPipe check. If `Task` were used for the same job (`@Body() dto: Task`), Nest would happily accept any garbage object at runtime — TypeScript's compile-time check doesn't reach into a live HTTP request body at all.

WHY YOU CAN'T JUST USE ONE FOR BOTH JOBS

```
Using `Task` (interface) as the @Body() type:
  - Compiles fine, LOOKS like it's validating — it isn't.
  - No decorators possible (interfaces are erased, so a decorator has
    no real class to attach itself to at runtime).
  - A client sending {"title": 123, "done": true, "id": 999} would be
    silently accepted — id and done should never be client-controlled.
```

Using `CreateTaskDto` (class) as the internal storage type: - Missing fields the internal model needs (`id` , `done` are absent from CreateTaskDto on purpose). - Mixes "what a client is allowed to send" with "what the app stores internally" — the moment those two need to diverge (and they almost always do — see id/done above), one class can't honestly represent both anymore.

SIDE-BY-SIDE TABLE

```
|                     | interface (Task)          | DTO (CreateTaskDto)       |
|---------------------|---------------------------|---------------------------|
| Represents          | internal/trusted data     | incoming/untrusted data   |
| Exists at runtime?  | No — fully erased         | Yes — a real class        |
| Can carry decorators?| No                        | Yes (@IsString, etc.)     |
| Validated by Nest?  | No                        | Yes, via ValidationPipe   |
| Typical location    | src/<feature>/interfaces/ | src/<feature>/dto/        |
| Typical use         | service return types,     | @Body()/@Query() params   |
|                     | array element types       |                           |
```

★ RULE OF THUMB

Ask: "does this shape need to survive into JavaScript and get checked against a live request?" If yes → DTO class. If it's just describing data your own code already trusts (service internals, return types) → interface. In this project: task.service.ts's internal `tasks: Task[]` array uses the interface; anything arriving through task.controller.ts's @Body() uses a DTO.

See also: theory/13-dto.txt, theory/12-interface.txt, theory/09-pipes.txt